Автономная некоммерческая организация
Дополнительного профессионального образования
Компьютерная Академия «ТОП» филиал «Академия ТОП Уфа»

Дипломная работа

по курсу «Автоматизация тестирования Python»

на тему: «Проектирование и реализация системы автоматизированного тестирования веб-интерфейса медицинского сайта “Alimma”»

Работа выполнена с помощью программ: Pycharm

Выполнил(а) студент(ка) Компьютерной Академии ТОП

Шавалеева Анастасия Александровна /_____
(ФИО) (подпись)

Дипломная работа допущена к защите и проверена на объем заимствования:

Директор филиала
АНО ДПО «Академия ТОП»

_____\ О.В. Комиссарова

Рук. учебной части филиала
АНО ДПО «Академия ТОП»
_____\ А.Н. Меркулова

Уфа, 2026г

СОДЕРЖАНИЕ

ВВЕДЕНИЕ..... 3

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ, ХАРАКТЕРИСТИКА САЙТА МЕДИЦИНСКОЙ КЛИНИКИ «ALIMMA» 6

1.1 Характеристика объекта тестирования- сайт «Alimma»..... 6

1.2. Тестирование программного обеспечения 6

1.3. Виды тестирования, применимые к сайту «Alimma»..... 7

1.4 Сравнение ручного и автоматизированного тестирования 9

1.5. Процесс тестирования и разработка тестовой документации..... 10

1.6 Необходимость автоматизации для сайта «Alimma» 12

ГЛАВА 2. ИНСТРУМЕНТЫ И ТЕХНОЛОГИИ ДЛЯ АВТОМАТИЗИРОВАННОГО ТЕСТИРОВАНИЯ 14

2.1 Выбор языка и фреймворков для автотестирования 14

2.2 Фреймворк для написания тестов: PyTest 15

2.3 Библиотеки для UI-тестирования: Selenium и Playwright..... 16

2.4 Дополнительные инструменты для разработки и отладки автотестов..... 17

2.5. Организация кода тестов: паттерн PageObject..... 18

ЗАКЛЮЧЕНИЕ 20

Приложение 1. Тест-кейсы для сайта “Alimma”……..22

Приложение 2. Реализация автотестов для сайта “Alimma”……30

ВВЕДЕНИЕ

Целью данной дипломной работы является разработка автоматизированных тестов для сайта медицинской клиники «Alimma» с использованием современных инструментов, что позволит повысить эффективность процесса тестирования и надежность продукта.

Выбор объекта исследования - сайта «Alimma» — обусловлен тем, что современные интернет-ресурсы, предоставляющие услуги, представляют собой сложные программные комплексы. Они включают в себя каталоги товаров, прайс-листы, расписание врачей, сложные системы фильтрации, корзину, личные кабинеты пользователей и многое другое. Сбой в любом из этих звеньев может разрушить всю цепочку предоставления услуг. Для пациентов сайт клиники является ключевым инструментом взаимодействия: от изучения направлений деятельности и знакомства с врачами до записи на приём. От безупречной работы ресурса напрямую зависит здоровье и комфорт клиента, а также репутация и прибыль клиники. Любой сбой может привести к тому, что пациент не получит своевременную помощь, а клиника потеряет доверие и доход.

Тестирование программного обеспечения, это не просто этап разработки, а критически важный процесс анализа программного продукта и сопутствующей документации, обеспечивающий надежность и функциональность продукта. Автоматизация тестирования позволяет проводить регулярные и глубокие проверки пользовательских сценариев.

Для сайта «Alimma», это возможность быстро и эффективно тестировать весь путь клиента: от поиска нужной услуги до подтверждения записи на прием. Автоматические тесты помогут убедиться, что формы обратной связи отправляются, информация о врачах и услугах корректно отображается, а процесс бронирования работает без ошибок. Это экономит ресурсы на многократные ручные проверки и гарантирует, что каждый посетитель получит качественный сервис.

Для реализации автоматизации выбран язык программирования Python. Он отличается лаконичностью и читаемостью кода, что позволяет писать тесты быстрее и проще их поддерживать. Богатая экосистема библиотек и фреймворков делает Python универсальным инструментом для разных видов тестирования. Фреймворк PyTest подходит для проверки бэкенда и API, а с помощью Playwright можно эмулировать действия пользователя в браузере и проверять корректность работы пользовательского интерфейса.

В первой главе рассмотрены теоретические основы тестирования программного обеспечения для сайта медицинской клиники: проанализированы особенности ручного и автоматизированного подходов, обоснована необходимость автоматизации для данного сайта, определены виды и методы тестирования, применимые к сайту «Alimma», а также описаны процессы разработки тестовой документации.

Вторая глава посвящена обоснованию выбора инструментов автоматизации. Для автоматизации тестирования сайта «Alimma» будет выбрано:

- Язык программирования: Python- простота, читаемость, большое количество библиотек.
- Фреймворк для тестов: PyTest- гибкость, фикстуры, параметризация, плагины.
- Библиотека для UI-тестирования: Playwright- стабильность, автоматические ожидания, кроссплатформенность, удобство отладки.
- Вспомогательный инструмент для разработки и отладки автотестов- DevTools, подбор уникальных и стабильных селекторов.

Данный набор инструментов полностью покрывает задачи проекта, обеспечивает высокую надёжность тестов и лёгкость их поддержки в дальнейшем.

В третьей главе будет описана практическая реализация тест-кейсов для сайта «Alimma».

В четвертой главе будет реализация автоматизированных тестов для сайта «Alimma»: написание кода, выполнения тестов и получение результатов после прогона.

В данной дипломной работе основное внимание уделяется автоматизированному функциональному тестированию методом «чёрного ящика», охватывающему ключевые сценарии регистрации, записи на приём и навигации по сайту «Alimma». Результаты работы демонстрируют, что автоматизация тестирования, способна решать реальные задачи бизнеса, повышать доверие пользователей, оптимизировать процесс контроля качества и обеспечивать уверенность в надёжности сайта, от которого зависят здоровье, спокойствие людей и прибыль для бизнеса.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ, ХАРАКТЕРИСТИКА САЙТА МЕДИЦИНСКОЙ КЛИНИКИ «ALIMMA»

1.1 Характеристика объекта тестирования- сайт «Alimma»

Сайт медицинской клиники «Alimma» представляет собой многофункциональный веб-ресурс, предназначенный для предоставления онлайн-услуг и информировании для пациентов. Основные разделы сайта включают: информацию о клинике, направления деятельности, список врачей с описанием квалификации, расписание приёма, прайс-лист на услуги, акции и новости, обратную связь, систему онлайн-записи на приём, а также личный кабинет пациента.

Для пользователя сайт является ключевым инструментом взаимодействия с клиникой: он должен обеспечивать удобный поиск нужной информации, быстрый доступ к расписанию, возможность записи к врачу без лишних сложных действий. Любая ошибка в работе этих функций может привести к потере клиента и доверия к самой клинике. Поэтому тестирование данного ресурса должно быть направлено на проверку как корректности отображения информации, так и стабильности работы всех сценариев пользователя.

1.2. Тестирование программного обеспечения

Тестирование - это процесс анализа программного продукта, направленный на выявление различий между реальным поведением системы и ожидаемым, описанным в требованиях. Основные цели тестирования: убедиться, что продукт работает согласно спецификации, проверить корректность выполнения всех заявленных функций, а также обнаружить ситуации, в которых поведение программы является ошибочным или не соответствует ожиданиям пользователя.

Для сайта «Alimma» цели тестирования:

- Проверить, что все страницы и разделы сайта загружаются корректно и отображают нужную информацию.
- Убедиться, что навигация по сайту интуитивно понятна и не вызывает затруднений.
- Проверить функциональность формы записи на приём: отправка данных, валидация полей, получение подтверждения.
- Оценить корректность работы системы фильтрации и поиска по врачам и услугам.
- Убедиться, что данные в личном кабинете пользователя отображаются верно.

1.3. Виды тестирования, применимые к сайту «Alimma»

В зависимости от целей и уровня доступа к системе различают множество видов тестирования. Для данного проекта будут использованы следующие:

По степени автоматизации:

- **Ручное тестирование** – метод проверки информационного продукта, когда специалист самостоятельно вручную проверяет программное обеспечение на соответствие требованиям заказчика. В данной работе у ручного тестирования, вспомогательная роль, используется на начальном этапе для первичного исследования функциональности, оценки удобства использования, проверки вёрстки и выявления неочевидных ошибок, которые сложно запрограммировать в автотестах.
- **Автоматизированное тестирование** –способ тестирования продукта с использованием программных средств (автотестов), которые выполняют шаги, заранее созданные человеком, автоматически. Автоматизация применима для регулярно повторяющихся сценариев, таких как: регистрация нового пользователя, авторизация, запись на приём к врачу, навигация по разделам сайта,поиск услуг, просмотр расписания, переход в личный кабинет. Автотесты позволят выполнять регрессионное тестирование после

каждого обновления сайта, гарантируя стабильность его работы. Именно этот подход, является ключевым в данной работе.

По целям:

- **Функциональное тестирование** — основной вид проверок для сайта «Alimma». Оно направлено на проверку того, что система выполняет нужные функции, охватывает все пользовательские сценарии: Регистрация нового пользователя и вход в личный кабинет, поиск врача, выбор даты и времени, заполнение формы, онлайн-запись, переходы между страницами, работа меню, фильтров и поиска.

По уровням:

- **Интеграционное тестирование** необходимо для проверки корректного взаимодействия различных компонентов сайта, например, фронтенда с бэкендом. На примере сайта, при успешной отправке формы, появится сообщение об успешной записи, это подтверждает то, что интеграция прошла успешно.
- **Системное тестирование** подразумевает проверку сайта как единого целого. Будет реализовано с помощью сквозных автоматизированных тестов (end-to-end), имитирующих действия реального пользователя от начала до конца сценария. Такие тесты охватывают регистрацию, запись и навигацию, проверяя сайт в комплексе.

По доступу к коду:

- **Метод чёрного ящика** является основным для тестирования сайта «Alimma», так как тестировщик взаимодействует с системой через интерфейс, не имея доступа к исходному коду. Этот метод применяется как при ручном, так и при автоматизированном тестировании (UI-тесты).

При автоматизации UI-тестов, необходимо учитывать как построены современные сайты.

Автотесты должны уметь корректно ожидать появления элементов, прежде чем взаимодействовать с ними. Многие элементы на странице могут

появляться или изменяться после загрузки, например: результаты поиска, выпадающие списки, сообщения об ошибках.

Тесты должны проверять не только видимые изменения, но и то, что данные действительно отправились и обработались, например: появление подтверждающего сообщения, такие действия происходят без перезагрузки страницы.

При разработке тестов важно выбирать максимально устойчивые селекторы, автотесты идентифицируют элементы на странице с помощью них, а при изменении вёрстки селекторы могут перестать работать, что приведёт к падению тестов.

Учитывать состояние элементов интерфейса, они могут находиться в разных состояниях: доступен/недоступен, видим/скрыт, отмечен/не отмечен. Тесты должны проверять не только наличие элемента, но и его корректное состояние в соответствии со сценарием.

1.4 Сравнение ручного и автоматизированного тестирования

Ручное тестирование остаётся незаменимым для исследования нового функционала, оценки удобства использования и поиска нестандартных ошибок, оно выполняет важные, но ограниченные задачи. Не требует навыков программирования, позволяет быстро проверить исправления и подходит для проектов с ограниченным бюджетом.

Для сайта «Alimma» ручное тестирование имеет некоторые недостатки:

- трудоёмко при выполнении регрессионных проверок,
- подвержено человеческому фактору,
- не дает возможность обеспечить регулярность и частоту прогонов,
- замедлит процесс выпуска новых версий сайта.

Автоматизированное тестирование решает эти проблемы и является ключевым инструментом обеспечения стабильности сайта «Alimma».

Преимущество автоматизированных тестов для сайта «Alimma»:

- выполняются быстрее ручных проверок,

- выполняют одни и те же шаги с одинаковой точностью, исключая риск пропуска ошибки из-за усталости или невнимательности человека. Важно для критических сценариев, например запись на прием, когда клиника может потерять клиента.
- возможность многократного выполнения с различными данными.
- возможность охватить большее количество сценариев и их варианты за то же время. Для сайта «Alimma» это возможность проверить все варианты записи, все пункты меню, все переходы.

В дипломной работе основное внимание уделяется автоматизированному тестированию как наиболее эффективному инструменту для регулярной проверки регистрации, записи на приём и навигации. Ручное тестирование используется вспомогательно для первичного исследования и оценки юзабилити, но основной объём проверок реализуется с помощью автотестов.

1.5. Процесс тестирования и разработка тестовой документации

Процесс тестирования включает несколько этапов:

1. Анализ требований и планирование: изучение функциональных возможностей сайта «Alimma», определение приоритетных сценариев для автоматизации, оценка объёма работ.
2. Разработка тестовой документации: создание тест-кейсов и чек-листов. Тест-кейс — это набор входных данных, шагов и ожидаемых результатов. Для сайта «Alimma» будут разработаны тест-кейсы для ключевых пользовательских сценариев- позитивные и негативные. Пример: тест-кейс «Успешная запись на приём к врачу» включает шаги: выбор услуги, выбор врача, выбор даты и времени, заполнение контактных данных, нажатие кнопки «Записаться», ожидаемый результат — появление сообщения об успешной записи.
3. Выполнение тестов: ручной прогон тест-кейсов и запуск автотестов.
4. Фиксация дефектов при обнаружении несоответствия между фактическим поведением сайта и ожидаемым результатом создаётся запись о

дефекте- баг-репорт. В работе для этих целей обычно используются специализированные системы (Jira, Trello, YouTrack).

5. После прогона автотестов обычно формируется отчёт, который позволяет оценить состояние продукта. Для этого используются, например Allure. Отчёт содержит информацию о количестве пройденных, упавших и пропущенных тестов, а также можно сделать скриншоты, видео для каждого упавшего теста, что облегчает анализ причин ошибок. На основе отчёта делаются выводы о стабильности протестированных сценариев и необходимости доработок. (Нужно ли писать ??)

Для создания тест-кейсов используются техники тест-дизайна, примеры их применение на сайте «Alimma»:

- **Эквивалентное разбиение** — Разделение входных данных на группы, которые должны обрабатываться одинаково. Значения внутри одной группы эквивалентны, они идентичны, но не одинаковы и приводят к одному и тому же результату. Достаточно проверить одно любое значение из класса. Например, для поля «телефон» можно выделить классы корректных номеров (10 цифр) и некорректных (буквы, недостаточная длина). Проверяется по одному представителю из каждого класса.

Корректные значения (имя из букв, с дефисом), некорректные значения (цифры, спецсимволы, пустое поле). Корректный email (с @), некорректный (без @, без точки, с пробелами), пустое поле, корректный номер (10 цифр), некорректный (меньше 10 цифр, больше 10 цифр, буквы, спецсимволы).

- **Анализ граничных значений** – когда нужна проверка крайних значений допустимых диапазонов, так как на них чаще возникают ошибки. Тестируются само граничное значение, значение перед границей и после нее. Минимальная длина (2 символа), максимальная длина (50 символов), значения на границе (1 символ, 51 символ). Для поля «Телефон» применить: 9 цифр, 10 цифр, 11 цифр.

- **Таблица принятия решений** -при тестирование сложных систем со множествами комбинаций условий. При всевозможных сочетаниях

нескольких параметров тестировщик получает разный результат, нужно подобрать нестандартные, редко встречающиеся сочетания, и убедиться, что система корректно их обрабатывает. Если форма записи имеет несколько условий, например: выбор врача, выбор времени, согласие на обработку данных, составляется таблица комбинаций для проверки всех вариантов.

- **Попарное тестирование** используется, когда большое количество комбинаций входных параметров и нужно сократить число тест-кейсов, выделяя только возможные пары значений. Например: при большом количестве параметров в фильтрах.

Эти техники помогут обеспечить высокое тестовое покрытие сайта «Alimma» при минимальном количестве тест-кейсов.

1.6 Необходимость автоматизации для сайта «Alimma»

Учитывая особенности сайта, автоматизация тестирования необходима по причинам:

- Ключевые сценарии, которые требуют регулярной проверки**.**

Регистрация пользователей, запись на приём и навигация по сайту, функции, от стабильности которых зависит удовлетворённость пациентов и доход клиники. Ручное выполнение таких проверок после каждого изменения в код отнимает значительное время тестировщика и замедляет процесс разработки.

- Сайт имеет множество взаимосвязанных функций, изменение в одном разделе, может повлиять на работу других. Автоматизированные тесты позволяют быстро и полностью проверить все ключевые сценарии после каждого изменения, минимизируя риск появления регрессионных дефектов.
- Автоматизация поможет сэкономить времени и ресурсы команды, так как тесты выполняются быстро, а ручные проверка тех же сценариев может занимать часы.
- Возможность запускать так часто, как это необходимо, хоть несколько раз в день.

- Автоматизация позволяет легко проверить один и тот же сценарий с множеством различных наборов данных.
- Гарантия стабильности. Сайт, который работает стабильно, где запись на приём всегда проходит без сбоев, а навигация интуитивно понятна, влияет на удовлетворённость пациентов и на репутацию и прибыль клиники.

Внедрение автоматизированного тестирования является обоснованным шагом для повышения качества и надёжности сайта медицинской клиники «Alimma».

У автоматизации тестирования много преимуществ, но все же есть свои недостатки, которые необходимо учитывать:

- Написание автотестов требует времени и квалификации, что увеличивает затраты. При частых изменениях интерфейса тесты приходится обновлять, что увеличивает стоимость их сопровождения.
- Тесты могут падать не из-за дефектов в продукте, а из-за проблем в самих тестах, например: нестабильные селекторы или ложные срабатывания -flaky tests. Тогда результаты прогона становятся недостоверными и требуют ручного анализа.
- Автотесты не могут проверить все, например: оценить удобство использования, визуальную привлекательность, интуитивность интерфейса, здесь требуются участие человека.

Для сайта «Alimma» плюсов от автоматизации ключевых сценариев перевешивает указанные недостатки, так как стабильность регистрации и записи критически важна для бизнеса.

ГЛАВА 2. ИНСТРУМЕНТЫ И ТЕХНОЛОГИИ ДЛЯ АВТОМАТИЗИРОВАННОГО ТЕСТИРОВАНИЯ

2.1 Выбор языка и фреймворков для автотестирования

Автоматизированное тестирование пользовательского интерфейса подразумевает написание программ, которые имитируют действия реального пользователя: клики по кнопкам, заполнение форм, переходы по ссылкам, проверку отображения элементов. Для этого необходим язык программирования и набор библиотек (фреймворк), обеспечивающих управление браузером и выполнение проверок.

Писать автоматизированные тесты можно на разных языках программирования: C#, Ruby, Kotlin, Java, Python и JavaScript. Каждый из них имеет свои особенности, но для данного проекта выбор сделан в пользу языка Python.

Python – отличается лаконичным и читаемым синтаксисом, что позволяет быстро писать и поддерживать тесты. Богатая экосистема включает фреймворк PyTest, лучший для тестирования, и современные библиотеки для управления браузером- Selenium, Playwright. Python прост в освоении, что хорошо для тестировщиков, не являющихся профессиональными разработчиками.

Преимущества Python:

- Тесты, написанные на Python, могут запускаться на любой операционной системе: Windows, Linux, macOS, без изменений.
- По автоматизации тестирования на Python существует огромное количество готовых примеров, библиотек и решений типовых проблем, что ускоряет разработку.
- PyTest и Playwright- основные инструменты проекта, разработаны с учётом особенностей Python и предоставляют удобные API именно для этого языка.

- При необходимости в проект можно легко добавить проверки баз данных, работу с файлами, генерацию отчётов и другие функции, оставаясь в рамках одного языка.

2.2 Фреймворк для написания тестов: PyTest

PyTest: Самый популярный фреймворк. Он прост, но мощен. Умеет запускать тесты параллельно, имеет множество плагинов, которые расширяют функционал, позволяют автоматизировать сложные задачи, такие как интеграция с базами или браузерами, понятный синтаксис проверок.

- **Простой синтаксис** – тесты оформляются как обычные функции с использованием встроенных проверок (assert).
- **Фикстуры (fixtures)** – удобны для подготовки окружения перед тестом, например: открытие браузера, авторизация пользователя или его очистки после выполнения.
- **Параметризация тестов** – позволяет запускать один и тот же тест с разными наборами входных данных, что особенно полезно для проверки различных вариантов заполнения форм.
- **Плагины** – расширяют функциональность: параллельный запуск тестов, генерация отчётов (Allure), интеграция с системами непрерывной интеграции.

Кроме базовых возможностей, PyTest есть маркировка **тестов** (@pytest.mark), которая позволяет группировать тесты по различным признакам, например: регистрация, запись, навигация и запускать только нужные группы, что очень удобно при разработке и отладке. Полезные плагины: pytest-xdist – для параллельного запуска тестов, что сокращает общее время прогона, pytest-html – для генерации простых HTML-отчётов, pytest-repeat – для повторного запуска упавших тестов, полезно при отладке нестабильных тестов.

Для всех автоматизированных тестов сайта «Alimma» PyTest будет использоваться PyTest.

2.3 Библиотеки для UI-тестирования: Selenium и Playwright

Для непосредственного управления браузером и взаимодействия с элементами веб-страниц используются специализированные библиотеки. Наиболее известные – Selenium и Playwright.

Selenium WebDriver – классический инструмент, поддерживающий все основные браузеры. Он имеет большое сообщество и множество примеров. Однако при работе с Selenium необходимо вручную настраивать ожидания загрузки элементов, что часто приводит к нестабильности тестов («flaky» тесты). Также требуется установка отдельных драйверов для каждого браузера.

Playwright – более современная библиотека, разработанная Microsoft. Её ключевые преимущества для UI-тестирования:

- Автоматическое ожидание (auto-waiting) – перед выполнением действия Playwright самостоятельно ждёт, когда элемент станет доступен для взаимодействия. Это значительно повышает стабильность тестов и упрощает код.
- Единый API для всех браузеров – не нужно устанавливать отдельные драйверы, Playwright поставляется со встроенными сборками Chromium, Firefox и WebKit.
- Работа с несколькими вкладками и контекстами – легко переключаться между вкладками, эмулировать мобильные устройства, географическое положение и сетевые условия.
- Встроенная поддержка скриншотов, видео и трассировки – упрощает отладку упавших тестов.
- Более высокая скорость выполнения по сравнению с Selenium.

Для сайта «Alimma», для которого важно проверить стабильность сценариев записи на приём и регистрации, поэтому выбором будет Playwright. Он позволит писать надёжные тесты, которые не будут падать из-за случайных задержек загрузки страниц, и обеспечит проверку в разных браузерах без дополнительных усилий.

2.4 Дополнительные инструменты для разработки и отладки автотестов

Помимо основных инструментов (Python, PyTest, Playwright), в процессе создания и поддержки автоматизированных UI-тестов активно используются встроенные средства браузеров – инструменты разработчика DevTools.

DevTools доступен во всех современных браузерах (Chrome, Firefox, Edge) и предоставляют незаменимую помощь при анализе структуры веб-страниц, подборе селекторов и отладке тестов.

Возможности DevTools применяемые при разработке автотестов для сайта «Alimma»:

- Вкладка Elements – позволяет просматривать и анализировать HTML-код страницы, изучать DOM-дерево, проверять CSS-стили элементов. Подобратья уникальные и стабильные селекторы-CSS-селекторы, XPath, для идентификации разных элементов: кнопок, полей ввода, ссылок. Можно тестировать селекторы прямо в браузере, чтобы убедиться, что они находят нужный элемент.
- Вкладка Console – отображает ошибки и предупреждения, которые могут влиять на работу интерфейса. Также в консоли можно выполнять произвольный JavaScript-код для проверки состояния страницы или поиска элементов, что помогает при отладке сложных сценариев.
- Вкладка Network – показывает все сетевые запросы, которые браузер отправляет при загрузке страницы и взаимодействии с ней. Для UI-тестирования это полезно при анализе того, какие данные передаются на сервер при отправке формы записи или регистрации, и для проверки корректности загрузки ресурсов. Вкладка помогает убедиться, что после клика на кнопку отправляется ожидаемый запрос и приходит успешный ответ.
- Вкладка Application – доступ к хранилищам браузера: LocalStorage, SessionStorage, Cookies, IndexedDB. Это важно для тестирования сценариев,

связанных с авторизацией и сохранением состояния пользователя: например, проверка наличия токена после входа в личный кабинет.

- Режим эмуляции устройств (Device Toolbar) – позволяет имитировать отображение сайта на различных мобильных устройствах и планшетах. Данный вид тестирования не рассматривается в данной работе, но в кроссбраузерном тестировании, данный режим помогает на этапе разработки тестов убедиться, что выбранные селекторы работают и на мобильных разрешениях.

Инструменты разработчика являются неотъемлемой частью рабочего процесса тестировщика-автоматизатора.

Использование DevTools значительно ускоряет процесс написания автотестов, снижает вероятность ошибок при выборе селекторов и помогает быстро разобраться в причинах падения тестов если, например изменился идентификатор элемента или не загрузился ресурс.

2.5. Организация кода тестов: паттерн PageObject

При написании автотестов важно выбрать инструменты и правильно организовать код, чтобы тесты были понятными, легко поддерживались и не дублировали друг друга. Для этого есть паттерн PageObject, позволяет создать отдельные классы для каждой страницы или значимого компонента**.**

Каждый такой класс инкапсулирует: элементы- кнопки, поля ввода, ссылки на странице; действия связанные с этими элементами- клик, ввод текста, выбор значения; проверки которые могут возвращать состояние элементов.

Преимущества, тесты работают не напрямую с селекторами и вызовами, а с методами PageObject-классов.

- Упрощение тестов. Тесты становятся короткими и читаемыми, так как скрывают детали реализации.

- Переиспользование кода. Если одна и та же страница используется в нескольких тестах, вся логика работы с ней содержится в одном месте.
- Упрощение поддержки. При изменении вёрстки достаточно обновить селекторы в одном классе PageObject, а не править много тестов.

В проекте для сайта «Alimma» планируется использовать паттерн PageObject. Будут созданы классы для основных страниц, пример: главная, страница регистрации, страница записи на приём, страница личного кабинета. Это обеспечит чистоту кода и лёгкость его поддержки в дальнейшем.

ЗАКЛЮЧЕНИЕ

Автоматизация является необходимым инструментом для обеспечения высокого качества цифровых продуктов. Ручное тестирование сохраняет свою актуальность для исследовательских и юзабилити-проверок, однако именно автоматизация позволяет эффективно справляться с рутинными регрессионными задачами, многократно ускорять проверку критических пользовательских сценариев и гарантировать стабильность работы сложных систем, особенно в условиях частых обновлений.

В ходе данной дипломной работы была исследована роль автоматизации тестирования в современной разработке программного обеспечения, а также продемонстрированы возможности её применения на примере веб-сайта медицинской клиники «Alimma».

Проведённое функциональное тестирование позволило проверить работоспособность ключевых модулей сайта, и выявить системные проблемы, которые напрямую влияют на качество обслуживания пациентов и бизнес-процессы клиники.

Наиболее критичной проблемой сайта оказалась некорректная работа валидации формы «Запись на приём». Сервер принимает и сохраняет заявки даже при отсутствии номера телефона, и при вводе любых произвольных символов в поле имени и нереальных дат. Это делает невозможным обратный звонок клиенту, что ведёт к потере потенциальных пациентов. Такие дефекты носят критический характер, так как это взаимодействие пользователя с клиникой.

Базовый функционал сайта в целом работает корректно. Все тесты навигации по разделам «Врачи», «Услуги» и «Цены», проверки отображения информации о специалистах, открытия чата мессенджера WhatsApp, отправки сообщений, работы пагинации и каруселей были пройдены успешно.

При этом нужно упомянуть, что тест клика по кнопке «Next» в пагинации был пройден и перелистывание страниц происходит, но в

интерфейсе данная кнопка не доработана: она визуально сливается с фоном и не содержит ни стрелки, ни текстовой подписи, что затрудняет её обнаружение пользователем и снижает удобство использования сайта.

При тестировании был выявлен дефект перехода на портал Prodoctorov: со страницы неврологии ссылка на отзывы о враче ведёт не на страницу сайта Prodoctorov, а на ту же страницу Невролог в Уфе, сайта “Alimma”. Это не позволяет потенциальным пациентам ознакомиться с реальными отзывами и рейтингом специалистов, что снижает уровень доверия к клинике, для медицинских услуг репутация врача является одним из ключевых факторов выбора. На странице неврологии обнаружена критическая проблема с вёрсткой: блоки информации наложены друг на друга, текст не читается, что делает невозможным получение пользователем сведений о направлениях лечения и методах работы.

В целом сайт функционален и пригоден к эксплуатации, но выявленные критические дефекты требуют незамедлительного исправления, так как влияют на репутацию учреждения. Проведённое тестирование подтвердило необходимость внедрения автоматизированных проверок регресса – особенно для сценариев с валидацией ввода, где ручные тесты могут быть недостаточно эффективны из-за человеческого фактора и ограниченного набора тестовых данных.

Задачи дипломной работы выполнены: исследована теоретическая база автоматизации тестирования, практически продемонстрированы её возможности и доказана целесообразность применения для повышения качества медицинского веб-сайта.

ПРИЛОЖЕНИЕ 1

Тест-кейсы для сайта медицинской клиники «Alimma»

Идентификатор	001
Приоритет теста	Высокий
Название теста	Загрузка главной страницы сайта.
Автор	Шавалеева А. А.
Предусловие	
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1. Ввести адрес сайта - https://alimma-ufa.ru	
2. Нажать Enter	2.Открылась главная страница сайта.
Результаты теста	Тест пройден

Идентификатор	002
Приоритет теста	Высокий
Название теста	Навигация по меню сайта, переход в раздел “Врачи”
Автор	Шавалеева А. А.
Предусловие	Открыть сайт Alimma – “https://alimma-ufa.ru”
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1. В хедере в верхнем меню кликнуть на пункт – “Врачи”.	1. Открылась страница со специалистами клиники, url адрес страницы изменился на – “https://alimma-ufa.ru/speczialisty-kliniki/”. 1.2 На странице заголовок “Специалисты клиники Alimma”.
Результаты теста	Тест пройден

Идентификатор	003
Приоритет теста	Высокий
Название теста	Навигация по меню сайта в раздел “Врачи” – “Ревматологи”
Автор	Шавалеева А. А.
Предусловие	Открыть сайт Alimma - “https://alimma-ufa.ru”
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат

1.В хедере в верхнем меню навести курсор на пункт – “Врачи”.	1.Открылся выпадающий список специальностей врачей.
2. Кликнуть на пункт выпадающего списка – “Ревматолог”.	2.Перешли на страницу врачей по специальности “Ревматолог”, url адрес страницы изменился. 2.1 На странице карточки докторов
Результаты теста	Тест пройден

Идентификатор	004
Приоритет теста	Средний
Название теста	Активность кнопки с цифрой 2 в пагинации на странице “Услуги”.
Автор	Шавалеева А. А.
Предусловие	Открыть сайт Alimma - “https://alimma-ufa.ru”
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1. В хедере в верхнем меню кликнуть на пункт – “Услуги”	1. Открылась страница услуг сайта. 1.1 Отображаются услуги на странице 1
2. Пролистать до пагинации	2. Кнопка “1” в цвете RGB# 0, 162, 156
3. Нажать на цифру 2	3.1Url адрес страницы изменился 3.2 Кнопка изменила цвет на RGB# 0, 162, 156 3.3 Список услуг на странице 2 изменился
Результаты теста	Тест пройден

Идентификатор	005
Приоритет теста	Средний
Название теста	Активность кнопки “Next” в пагинации на странице “Услуги”.
Автор	Шавалеева А. А.
Предусловие	Открыть сайт Alimma - “https://alimma-ufa.ru”
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1. В хедере в верхнем меню кликнуть на пункт – “Услуги”	1. Открылась страница услуг сайта. 1.1 Отображаются услуги на странице 1
2. Пролистать до пагинации	2. Кнопка с цифрой 1 в цвете RGB# 0, 162, 156
3. Нажать кнопку Next	3. Кнопка с цифрой 2 изменила цвет на RGB# 0, 162, 156 3.2 Список услуг на странице 2 изменился 3.3 Появилась кнопка для перелистывания назад “Prev”
Результаты теста	Тест пройден

Идентификатор	006
Приоритет теста	Высокий
Название теста	Отображение цен на услуги, категории “Ревматолог”
Автор	Шавалеева А. А.
Предусловие	Открыть сайт “Alimma - https://alimma-ufa.ru”
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1. В хедере в верхнем меню кликнуть на пункт – “Цены”	1. Открылась страница сайта, url адрес изменился на– https://alimma-ufa.ru/price/ 1.2 Заголовок страницы – “Цены на медицинские услуги”.
2. Выбрать раздел – “Ревматолог”	2.На странице таблица с полями: Код услуги, Наименование, Стоимость (руб.)
Результаты теста	Тест пройден

Идентификатор	007
Приоритет теста	Средний
Название теста	Переход на главную страницу сайта по логотипу в шапке сайта.
Автор	Шавалеева А. А.
Предусловие	Открыть сайта Alimma, страницу услуг- “https://alimma-ufa.ru/uslugi/”
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1. Кликнуть на логотип компании “Alimma Клиника современной медицины”	1. Перешли на главную страницу сайта, url адрес изменился на- “https://alimma-ufa.ru/”
Результаты теста	Тест пройден

Идентификатор	008
Приоритет теста	Высокий
Название теста	Открытие чата мессенджера “WhatsApp”
Автор	Шавалеева А. А.
Предусловие	Открыть сайт Alimma - “https://alimma-ufa.ru”
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат

1. В правом нижнем углу кликнуть иконку Мессенджера “WhatsApp”	1.Открывается окно чата, которое содержит: Заголовок- "Алимма" Статус- "Online" Текстовое сообщение – “ Мы здесь, чтобы помочь. Пишите нам WhatsApp по любым вопросам” Поле для ввода текста с плейсхолдером- “Здравствуйте!” Кнопку закрытия, Кнопку отправки сообщения
Результаты теста	Тест пройден

Идентификатор	009
Приоритет теста	Высокий
Название теста	Отправка сообщения через мессенджер “WhatsApp”
Автор	Шавалеева А. А.
Предусловие	Открыть сайт Alimma - “https://alimma-ufa.ru”
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1. В правом нижнем углу кликнуть иконку Мессенджера “WhatsApp”	1.Открывается окно чата, которое содержит: Заголовок- "Алимма" Статус- "Online" Текстовое сообщение – “ Мы здесь, чтобы помочь. Пишите нам WhatsApp по любым вопросам” Поле для ввода текста с плейсхолдером- “Здравствуйте!” Кнопку закрытия, Кнопку отправки сообщения
2.Ввести текст "Нужна консультация по услуге"	
3.Нажать на иконку “Самолетик” для отправки сообщения	3.Открылась новая ссылка Url изменился
Результаты теста	Тест пройден

Идентификатор	010
Приоритет теста	Высокий
Название теста	Просмотр анкеты врача
Автор	Шавалеева А. А.
Предусловие	Перейти по ссылке - https://alimma-ufa.ru/medilink_doctor_category/revmatolog
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1.Нажать на кнопку “Обо Мне” в анкете врача-Каримова Лилия Ильсуровна	1. Перешли на страницу, url адрес изменился на - https://alimma-ufa.ru/doctors/salimyanov-rashid-gataevich-2/ 1.2 Заголовок страницы – Каримова Лилия Ильсуровна 1.3 Кнопка – Запись на прием. 1.4 Анкета врача содержит информацию: Обо Мне, Дипломы И Сертификаты 1.5 Таблицу (год, степень, университет)
Результаты теста	Тест пройден

Идентификатор	011
Приоритет теста	Средний
Название теста	Навигация по сайту, через карусель “Услуг” на главной странице
Автор	Шавалеева А. А.
Предусловие	Открыть сайт “Alimma - https://alimma-ufa.ru”
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1. На главной странице пролистать до карусели “Услуги”	1.Отображается заголовок раздела “Услуги”
2. Нажать на стрелку в правой стороне для пролистывания услуг.	2.Отобразилась новая услуга в карусели
3. Нажать на карточку – “Лечение боли в голеностопном суставе”	3. Перешли на страницу со статьей url адрес изменился - https://alimma-ufa.ru/uslugi/travmatologiya/bol-v-golenostopnom-sustave/
Результаты теста	Тесте пройден

Идентификатор	012
Приоритет теста	Средний
Название теста	Переход на портал Prodoctorov, со страницы Неврология
Автор	Шавалеева А. А.
Предусловие	Перейти по ссылке https://alimma-ufa.ru/uslugi/nevrologiya/
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1.Пролистать страницу до заголовка– Врач-невролог клиники Alimma.	
2. Нажать на ссылку Prodoctorov, в карточке врача -Тихонова Ольга Ивановна.	2. Перешли на сайт Prodoctorov - https://prodoctorov.ru/ufa/lpu/70513-alimma/
Результаты теста	Тесте не пройден (Открылась новая вкладка сайта страницы Невролог в Уфе, без перехода на сайт Prodoctorov)

Идентификатор	013
Приоритет теста	Высокий
Название теста	Запись на прием к врачу, отправка формы с валидными данными со страницы Неврология.
Автор	Шавалеева А. А.
Предусловие	Перейти по ссылке- https://alimma-ufa.ru/uslugi/nevrologiya/
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1.Скролить страницу до заголовка - Врач-невролог клиники Alimma	1. Отобразилась карточка врача
2. Нажать кнопку “Записаться к врачу”, под анкетой врача- Тихонова Ольга Ивановна	2. Открылось модальное окно “Запись на прием”
3. Ввести данные: Ваше имя и фамилия: Иван Кукаревич Ваш номер телефона: 9177771122 Дата и запись на прием: 2026-04-29 Услуга, которую хотите получить (Комментарий): Нужно консультация ревматолога на выбранную дату.	
3. Нажать на кнопку “Записаться”.	3. Появляется уведомление- “Запись на прием

	Ваша заявка успешно отправлена! Наш менеджер скоро с вами свяжется.”
Результаты теста	Тест пройден

Идентификатор	014
Приоритет теста	Высокий
Название теста	Запись на прием к врачу, отправка незаполненной формы.
Автор	Шавалеева А. А.
Предусловие	Перейти по ссылке-https://alimma-ufa.ru/uslugi/nevrologiya/
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1. Нажать на кнопку “Запись на прием”, под анкетой врача.	1.Открылось модальное окно “Запись на прием”
2. Нажать на кнопку “Записаться”	2.Под полями ввода: Ваше имя и фамилия, Ваш номер телефона, Дата записи на прием, отобразились уведомления: "Это поле является обязательным."
Результаты теста	Тест не пройден (Под полем ввода: Ваш номер телефона, не отобразилось уведомление, менеджер не сможет связаться.)

Идентификатор	015
Приоритет теста	Высокий
Название теста	Запись на прием к врачу, отправка формы с не валидными данными.
Автор	Шавалеева А. А.
Предусловие	Перейти по ссылке-https://alimma-ufa.ru/uslugi/nevrologiya/
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1. Нажать на кнопку “Запись на прием”, под анкетой врача.	1. Открылось модальное окно “Запись на прием”
2. Ввести данные: Ваше имя и фамилия: “Зу12Ru” Ваш номер телефона: “ 00110011001“	

Дата и запись на прием: “11111-11-11” Услуга которую хотите получить (Комментарий): “You need записаться a rheumatologist %& прием 275760 dates.”	
3. Нажать на кнопку “Записаться”	3.Под полями ввода отобразились уведомления: Ваше имя и фамилия- “Введены некорректные данные” Ваш номер телефона –“Введён некорректный телефонный номер.” Дата записи на прием, “Введена некорректная дата.”
Результаты теста	Тест не пройден (Заявка успешно отправлена не отобразились уведомления о некорректно введенных данных)

Идентификатор	016
Приоритет теста	Высокий
Название теста	Запись на прием к врачу, отправка формы с неполными данными
Автор	Шавалеева А. А.
Предусловие	Перейти по ссылке-https://alimma-ufa.ru/uslugi/nevrologiya/
ОС	Windows 11
Браузер	Chrome
Шаги выполнения	Ожидаемый результат
1. Нажать на кнопку “Запись на прием”, под анкетой врача.	1.Открылось модальное окно “Запись на прием”
2. Ввести данные в поле данные: Ваше имя и фамилия: “,Mbuygt76”	
3. Нажать на кнопку “Записаться”	3. Под полями ввода: Ваш номер телефона, Дата записи на прием, отобразилось уведомление: “Это поле является обязательным.”
Результаты теста	Тест не пройден (Заявка успешно отправлена, не введен номер телефона менеджер не сможет связаться)

ПРИЛОЖЕНИЕ 2

Реализация автоматизированных тестов для сайта “Alimma”

Tests

test_01_open_main_page.py

```
import allure
from playwright.sync_api import Page, expect
from pages.main_page import MainPage


@allure.title("Загрузка главной страницы сайта Alimma.")
@allure.description("Главная страница успешно загружается, в шапке сайта логотип
компании.")
def test_open_maim_page(page: Page):
    main_page = MainPage(page)

    with allure.step("Открытие главной страницы сайта."):
        main_page.open()

    with allure.step("Проверка URL и логотипа главной страницы"):
        expect(page).to_have_url("https://alimma-ufa.ru/")

        header = main_page.get_header()
        expect(header.logo.wrapper).to_be_visible()
```

test_02_navigation_doctor.py

```
import allure
from playwright.sync_api import Page, expect
from pages.main_page import MainPage
from pages.page_speczialisty import PageSpeczialisty


@allure.title("Навигация по меню, переход в раздел- Врачи.")
@allure.description("Тестирование главного меню, проверки URL, наличие заголовка на
странице")
def test_navigation_doctor(page: Page):
    main_page = MainPage(page)
    main_page.open()

    with allure.step("Кликнуть на пункт меню 'Врачи'"):
        header = main_page.get_header()
        header.click_menu_link("Врачи")

    with allure.step("Проверка Url адреса страницы"):
        speczialisty = PageSpeczialisty(page)
        expect(page).to_have_url(speczialisty.url)

    with allure.step("Проверка заголовка страницы'"):
        speczialisty.check_title_h1("Специалисты клиники Alimma")
```

test_03_navigation_doctor_revmatolog.py

```
import allure
from playwright.sync_api import Page, expect
from pages.main_page import MainPage
from pages.page_revmatolog import PageRevmatolog


@allure.title("Навигация через выпадающее подменю, переход к разделу Врачи-
Ревматолог")
@allure.description("Переход на страницу через подменю, проверка изменения URL,
заголовка, отображение карточки врача")
def test_navigation_doctor_revmatolog(page: Page):
    main_page = MainPage(page)
```

```
    main_page.open()

    with allure.step("Навести курсор на пункт меню 'Врачи'"):
        header = main_page.get_header()
        header.hover_menu("Врачи")

    with allure.step("Кликнуть на подпункт меню 'Ревматолог'"):
        header.click_submenu("Врачи", "Ревматолог")
        revmatolog = PageRevmatolog(page)

    with allure.step("Проверка Url адреса страницы и заголовка"):
        expect(page).to_have_url(revmatolog.url)
        revmatolog.check_title_h1("Ревматолог")

    with allure.step("Проверка отображения карточек докторов"):
        expect(revmatolog.get_doctor_cards().first).to_be_visible()
```

test_04_pagination_count.py

```
import allure
from playwright.sync_api import Page, expect
from pages.main_page import MainPage
from pages.page_uslugi import PageUslugi


@allure.title("Пагинация на странице Услуги, активность кнопки и смена контента")
@allure.description("Активность кнопки с цифрами в пагинации, смена URL, изменение
цвета активной кнопки")
def test_pagination_count(page: Page):
    main_page = MainPage(page)
    main_page.open()

    with allure.step("Кликнуть на пункт меню 'Услуги'"):
        header = main_page.get_header()
        header.click_menu_link("Услуги")

    with allure.step("Проверка Url адреса страницы "):
        uslugi = PageUslugi(page)
        expect(page).to_have_url(uslugi.url)

    with allure.step("Проверка заголовок страницы H1"):
        uslugi.check_title_h1("Услуги")

    with allure.step("Проверка отображения услуг на странице 1"):
        expect(uslugi.get_uslugi().first).to_be_visible()
        first_service_text_page1 = uslugi.get_uslugi().first.inner_text()

    with allure.step("Проверка отображения активной кнопки 1 в пагинации"):
        active_button = uslugi.pagination.get_active_button()
        expect(active_button).to_have_css("background-color", "rgb(0, 162, 156)")

    with allure.step("Кликнуть на цифру 2 в пагинации"):
        uslugi.click_number(2)

    with allure.step("Проверка Url адреса после перехода на страницу 2"):
        expect(page).to_have_url("https://alimma-ufa.ru/uslugi/page/2/")

    with allure.step("Проверка отображения активной кнопки 2"):
        active_button_after = uslugi.pagination.get_active_button()
        expect(active_button_after).to_have_css("background-color", "rgb(0, 162,
156)")

    with allure.step("Проверка изменения списка услуг на странице"):
        expect(uslugi.get_uslugi().first).to_be_visible()
        expect(uslugi.get_uslugi().first).not_to_have_text(first_service_text_page1)
```

test_05_pagination_next.py

```
import allure
```

```
from playwright.sync_api import Page, expect
from pages.main_page import MainPage
from pages.page_uslugi import PageUslugi


@allure.title("Пагинация кнопки Вперед- Next")
@allure.description("Активность кнопки Next в пагинации, смена URL, изменение цвета
активной кнопки, изменение "
                    "контента на странице")
def test_pagination_next(page: Page):
    main_page = MainPage(page)
    uslugi = PageUslugi(page)
    main_page.open()

    with allure.step("Кликнуть на пункт меню 'Услуги'"):
        header = main_page.get_header()
        header.click_menu_link("Услуги")

    with allure.step("Проверка Url страницы Услуг"):
        expect(page).to_have_url(uslugi.url)

    with allure.step("Проверика заголовок страницы H1"):
        uslugi.check_title_h1("Услуги")

    with allure.step("Проверка отображения услуг на странице 1"):
        names_page1 = uslugi.get_name_uslugi()

    with allure.step("Проверка цвета активной кнопки пагинации (страница 1)"):
        active_button = uslugi.pagination.get_active_button()
        expect(active_button).to_have_css("background-color", "rgb(0, 162, 156)"),
f"Цвет активной кнопки не: {active_button}"

    with allure.step("Нажать на кнопку Next"):
        uslugi.pagination.click_button_next()

    with allure.step("Проверка видимость кнопки Prev"):
        expect(uslugi.pagination.get_button_prev()).to_be_visible()

    with allure.step("Проверка URL после перехода на страницу 2"):
        expect(page).to_have_url("https://alimma-ufa.ru/uslugi/page/2/")

    with allure.step("Проверка номер активной кнопки в пагинации"):
        assert uslugi.pagination.get_actual_number() == 2, "Активная страница не 2"

    with allure.step("Проверка изменения услуг на странице 2"):
        names_page2 = uslugi.get_name_uslugi()
        assert names_page1 != names_page2, "Списки услуг на первой и второй странице
одинаковы"

        active_button_page2 = uslugi.pagination.get_active_button()
        expect(active_button_page2).to_have_css("background-color", "rgb(0, 162,
156)"), f"Цвет активной кнопки на странице 2 не: {active_button}"

    with allure.step("Проверка отображения кнопки 'Previous'"):
        expect(uslugi.pagination.get_button_prev()).to_be_visible()
```

test_06_price_services.py

```
import allure
from playwright.sync_api import Page, expect
from pages.main_page import MainPage
from pages.page_price_uslug import PriceUslug


@allure.title("Страница цен, отображение таблицы Ревматолог")
@allure.description("Тестирование страницы Цены, отображение таблицы раздел
Ревматолог")
def test_prices_for_services(page: Page):
    main_page = MainPage(page)
    price = PriceUslug(page)
```

```
    main_page.open()

    with allure.step("Кликнуть на пункт меню 'Цены'"):
        header = main_page.get_header()
        header.click_menu_link("Цены")

    with allure.step("Проверка Url страницы "):
        expect(page).to_have_url(price.url)

    with allure.step("Проверка заголовка страницы "):
        price.check_title_h1("Цены на медицинские услуги")

    with allure.step("Кликнуть по разделу 'Ревматолог'"):
        price.click_section("Ревматолог")
         page.wait_for_timeout(2000)
        assert price.get_table(), "Таблица с ценами не видима"

    with allure.step("Проверка наличия услуги по коду "):
        data_code = price.get_table_code("B01.040.001")
        expect(data_code).to_be_visible()

    with allure.step("Проверка наличия услуги по названию "):
        data_name = price.get_table_name_uslugi(
            "Первичный прием (осмотр, консультация) врача ревматолога 1 категории, с
выдачей заключения на руки")
        expect(data_name).to_be_visible()

    with allure.step("Проверка цены услуги с определенным кодом "):
        price_cell = price.get_price_by_code("B01.040.001")
        expect(price_cell).to_have_text("3 300")
```

test_07_navigate_by_logo.py

```
import allure
from playwright.sync_api import Page, expect
from pages.page_uslugi import PageUslugi


@allure.title("Переход на главную страницу через логотип")
@allure.description("Тестирование логотипа, преход на главную страницу сайта")
def test_navigate_by_logo(page: Page):
    uslugi_page = PageUslugi(page)
    uslugi_page.open()

    with allure.step("Кликнуть на логотип компании"):
        uslugi_page.header.click_logo()

    with allure.step("Проверка Url страницы"):
        expect(page).to_have_url("https://alimma-ufa.ru/")
```

test_08_messenger_whatsApp.py

```
import allure
from playwright.sync_api import Page, expect
from pages.main_page import MainPage
from components.window_messenger import WindowMessager


@allure.title("Открытие чата WhatsApp")
@allure.description("Тестирование Мессенджера WhatsApp, открытие окна чата, наличие
атрибутов")
def test_open_messenger_whatsApp(page: Page):

    main_page = MainPage(page)
    window_messenger = WindowMessager(page)
    main_page.open_main_page()

    with allure.step("Кликнуть на иконку Мессендера WhatsApp"):
        main_page.messenger_link.get_link_messenger()
        main_page.click_link_messenger()
```

```
    with allure.step("Проверка заголовка"):
        expect(window_messenger.get_title()).to_have_text("Алимма")

    with allure.step("Проверка статуса"):
        expect(window_messenger.get_status()).to_contain_text("Online")

    with allure.step("Проверка отображения текстового сообщения приветствия "):
        expect(window_messenger.get_text_in_window()).to_have_text("Мы здесь, чтобы
помочь. Пишите нам WhatsApp по любым вопросам")

    with allure.step("Проверка отображения поля ввода с плейсхолдером"):
        input_field = window_messenger.get_input()
        expect(input_field).to_have_value("Здравствуйте!")

    with allure.step("Проверка отображения кнопок: закрытие и отправка"):
        expect(window_messenger.get_send_button()).to_be_visible()
        expect(window_messenger.get_button_close()).to_be_visible()
```

test_09_send_a_messenger_whatsApp.py

```
import re

import allure
from playwright.sync_api import Page, expect
from pages.main_page import MainPage
from components.window_messenger import WindowMessager


@allure.title("Отправка сообщений WhatsApp")
@allure.description("Тестирование Мессенджера WhatsApp, отправка сообщения")
def test_send_a_messanger(page: Page):
    main_page = MainPage(page)
    window_messenger = WindowMessager(page)
    main_page.open_main_page()

    with allure.step("Кликнуть на иконку Мессендера WhatsApp"):
        main_page.messenger_link.get_link_messenger()
        main_page.click_link_messenger()

    with allure.step("Проверка заголовка"):
        expect(window_messenger.get_title()).to_have_text("Алимма")

    with allure.step("Проверка статуса"):
        expect(window_messenger.get_status()).to_contain_text("Online")

    with allure.step("Проверка отображения текстового сообщения, приветствия"):
        expect(window_messenger.get_text_in_window()).to_have_text("Мы здесь, чтобы
помочь. Пишите нам WhatsApp по любым вопросам")

    with allure.step("Проверка отображения поля ввода с плейсхолдером"):
        input_field = window_messenger.get_input()
        expect(input_field).to_have_value("Здравствуйте!")

    with allure.step("Проверка отображения кнопок: закрытие и отправка"):
        expect(window_messenger.get_send_button()).to_be_visible()
        expect(window_messenger.get_button_close()).to_be_visible()

    with allure.step("Ввести новый текст"):
        new_text = "Нужна консультация по услуге"
        window_messenger.clear_message(new_text)
        expect(window_messenger.get_input()).to_have_value(new_text)

    with allure.step("Кликнуть на иконку самолетик, для отправки сообщения"):
        with page.context.expect_page() as new_page_info:
            window_messenger.click_button_send()
        new_page = new_page_info.value
        new_page.close()
        # expect(new_page).to_have_url(re.compile(r"^https://api\.whatsapp\.com/"))
```

test_10_profiles_doctors.py

```
import allure
from playwright.sync_api import Page, expect
from pages.page_revmatolog import PageRevmatolog


@allure.title("Открытие профиля доктора на странице Ремватолог")
@allure.description("Отображение данных о враче: описание обо мне, информацию об
учебе.")
def test_profile_doctor(page: Page):
    page_revmatolog = PageRevmatolog(page)
    page_revmatolog.open()

    with allure.step("Кликнуть - Обо мне, врача Каримова Лилия."):
        profile = page_revmatolog.click_about_me_doctor("Каримова Лилия Ильсуровна")

    with allure.step("Проверка Url страницы "):
        expect(page).to_have_url("https://alimma-ufa.ru/doctors/karimova-liliya-
ilsurovna/")

    with allure.step("Отображение заголовка страницы"):
        profile.check_title_h1("Каримова Лилия Ильсуровна")

    with allure.step("Отображение заголовка анкеты Обо Мне"):
        expect(profile.get_title_about_me()).to_be_visible()

    with allure.step("Отображение заголовка таблицы Дипломы И Сертификаты"):
        expect(profile.get_title_diplom()).to_be_visible()
        table = profile.diplom_table()
        # expect(table).to_have_count(2)

    with allure.step("Проверка отображения данных в таблице: год, степерь,
университет"):
        expect(profile.diplom_table_year(0)).to_contain_text("24.06.2019")
        expect(profile.diplom_table_stepen(0)).to_contain_text("Диплом специалиста")
        expect(profile.diplom_table_university(0)).to_contain_text("Башкирский
государственный медицинский университет")

    with allure.step("Проверка отображения кнопки 'Записаться на прием'"):
        expect(profile.get_appointment_button()).to_be_visible()
```

test_11_navigation_to_services_in_carousel.py

```
import allure
from playwright.sync_api import Page, expect
from pages.main_page import MainPage
from components.carusel_uslugi import CaruselUslugi
from pages.page_bol_golenostopnom_sustave import RevmatologGolenostop


@allure.title("Карусель услуги, навигация в раздел")
@allure.description("Тестирование навигации в услуги через карусель на главной
странице сайта ")
def test_navigation_to_services_in_carousel(page: Page):
    main_page = MainPage(page)
    carusel = CaruselUslugi(page)
    main_page.open()

    with allure.step("На галавной странице сайта пролистать до карусели 'Услуги' "):
        carusel.carusel.scroll_into_view_if_needed()
        expect(carusel.carusel).to_be_visible()

    with allure.step("Проверка заголовка раздела "):
        main_page.check_title_h2("Услуги")

    with allure.step("Нажать на стрелку в правой стороне до появления нужной
услуги:'Лечение боли в голеностопном "
                     "суставе'"):
        get_title = "Лечение боли в голеностопном суставе"
        found = carusel.scroll_to_item_by_title(get_title)
```

```
    with allure.step("Проверка что отобразилась нужная услуга"):
        assert found, f"Карточка с названием '{get_title}' не найдена"

    with allure.step("Нажать на карточку – "Лечение боли в голеностопном суставе"):
        carusel.click_item_by_title(get_title)

    with allure.step("Проверка Url адреса, перехода на новую страницу"):
        expected_url = RevmatologGolenostop(page).url
        expect(page).to_have_url(expected_url)
```

test_12_switch_in_portal_prodoctorov.py

```
import allure
from playwright.sync_api import Page, expect
from components.cart_profile_doctor import CardProfileDoctor
from controls.link_prodoctorov import LinkProdoctorov
from pages.main_page import MainPage
from pages.page_nevrolog import PageNevrolog
from pages.page_prodoctorov import PageProdoctorov


@allure.title("Переход на портал Prodoctorov, со страницы Неврология")
@allure.description("Переход на портал Prodoctorov по ссылке под карточкой врача")
def test_switch_in_portal_prodoctorov(page: Page):

    nevrolog_page = PageNevrolog(page, "Врач-невролог клиники Alimma")
    doctor_name = "Тихонова Ольга Ивановна"
    doctor_card = CardProfileDoctor(page, doctor_name)
    nevrolog_page.open()

    with allure.step("Пролистать статью до заголовка - Врач-невролог клиники Alimma"):
        nevrolog_page.scroll_to_section()

    with allure.step("Проверка отображения заголовка и карточки врача "):
        expect(nevrolog_page.section).to_have_text("Врач-невролог клиники Alimma")
        expect(doctor_card.wrapper).to_be_visible()

    with allure.step("Нажать на ссылку Prodoctorov в карточке врача"):
        prodoctorov_link = LinkProdoctorov(page, doctor_card.wrapper, "Prodoctorov")
        with page.context.expect_page() as new_page_info:
            prodoctorov_link.click()

    with allure.step("Проверка Url страницы"):
        new_page = new_page_info.value
        prodoctorov_page = PageProdoctorov(new_page)
        prodoctorov_page.check_is_prodoctorov()
```

test_13_recording_a_doctor_valid_data.py

```
import allure
from playwright.sync_api import Page, expect
from components.cart_profile_doctor import CardProfileDoctor
from components.form_recording_a_doctor import FormRecording
from pages.page_nevrolog import PageNevrolog


@allure.title("Запись на приём на странице Неврология: отправка формы с валидными
данными")
@allure.description("Форма записи отправляется, высвечивается сообщение: "Ваша заявка
успешно отправлена! Наш "
                    "менеджер скоро с вами свяжется" ")
test_recording_a_doctor_valid_data(page: Page):
    nevrolog_page = PageNevrolog(page, "Врач-невролог клиники Alimma")
    doctor_name = "Тихонова Ольга Ивановна"
    doctor_card = CardProfileDoctor(page, doctor_name)
    form = FormRecording(page)
    nevrolog_page.open()

    with allure.step("Пролистать до секции 'Врач-невролог клиники Alimma'"):
        nevrolog_page.scroll_to_section()
```

```
    with allure.step("Проверка отображения заголовка и карточки врача "):
        expect(nevrolog_page.section).to_have_text("Врач-невролог клиники Alimma")
        expect(doctor_card.wrapper).to_be_visible()

    with allure.step("Нажать кнопку “Записаться к врачу”, под анкетой врача"):
        doctor_card.click_recording_button()

    with allure.step("Проверка заголовка формы"):
        expect(form.title_form).to_have_text("Запись на прием")

    with allure.step("Ввести данные"):
        form.fill_input_name("Иван Кукаревич")
        form.fill_input_telephone("9177771122")
        form.fill_input_date("2026-04-29")
        form.fill_input_comment("Нужна консультация ревматолога на выбранную дату.")

    with allure.step("Нажать на кнопку- “Записаться””):
        form.click_button_recording()

    with allure.step("Проверка уведомления об успешной записи"):
        form.get_success_message()
```

test_14_recording_a_doctor_empty_data.py

```
import allure
from playwright.sync_api import Page, expect
from components.cart_profile_doctor import CardProfileDoctor
from components.form_recording_a_doctor import FormRecording
from pages.page_nevrolog import PageNevrolog


@allure.title("Запись на приём на странице Неврология: отправка формы с незополненными
данными")
@allure.description("Форма записи не отправляется, под полями ввода появляются
уведомления: “Это поле является "
                    "обязательным”")
def test_recording_a_doctor_empty_data(page: Page):
    nevrolog_page = PageNevrolog(page, "Врач-невролог клиники Alimma")
    doctor_name = "Тихонова Ольга Ивановна"
    doctor_card = CardProfileDoctor(page, doctor_name)
    form = FormRecording(page)
    nevrolog_page.open()

    with allure.step("Пролистать статью до секции- Врач-невролог клиники Alimma"):
        nevrolog_page.scroll_to_section()

    with allure.step("Проверка отображения заголовка и карточки врача "):
        expect(nevrolog_page.section).to_have_text("Врач-невролог клиники Alimma")
        expect(doctor_card.wrapper).to_be_visible()

    with allure.step("Нажать кнопку “Запись на прием”, под анкетой врача"):
        doctor_card.click_recording_button()

    with allure.step("Проверка заголовка формы"):
        expect(form.title_form).to_have_text("Запись на прием")

    with allure.step("Не вводить данные, нажать на кнопку 'Записаться'"):
        form.fill_input_name("")
        form.fill_input_telephone("")
        form.fill_input_date("")
        form.fill_input_comment("")
        form.click_button_recording()

    with allure.step("Проверка уведомлений о необходимости заполнить данные имени и
фамилии"):
        expect(form.get_name_error()).to_have_text("Это поле является обязательным.")

    with allure.step("Проверка уведомлений о необходимости заполнить данные
телефонного номера"):
```

```
        expect(form.get_telephone_error()).to_have_text("Это поле является
обязательным.")

    with allure.step("Проверка уведомлений о необходимости заполнить данные записи на
прием"):
        expect(form.get_date_error()).to_have_text("Это поле является обязательным.")
```

test_15_recording_a_doctor_invalid_data.py

```
import allure
from playwright.sync_api import Page, expect
from components.cart_profile_doctor import CardProfileDoctor
from components.form_recording_a_doctor import FormRecording
from pages.page_nevrolog import PageNevrolog


@allure.title("Запись на приём на странице Неврология: отправка формы с невалидными
данными")
@allure.description("Форма записи не отправляется, появляются сообщения о некорректных
данных")
def test_recording_a_doctor_invalid_data(page: Page):
    nevrolog_page = PageNevrolog(page, "Врач-невролог клиники Alimma")
    doctor_name = "Тихонова Ольга Ивановна"
    doctor_card = CardProfileDoctor(page, doctor_name)
    form = FormRecording(page)
    nevrolog_page.open()

    with allure.step("Пролистать статью до секции- Врач-невролог клиники Alimma"):
        nevrolog_page.scroll_to_section()

    with allure.step("Проверка отображения заголовка и карточки врача "):
        expect(nevrolog_page.section).to_have_text("Врач-невролог клиники Alimma")
        expect(doctor_card.wrapper).to_be_visible()

    with allure.step("Нажать кнопку “Запись на прием”, под анкетой врача"):
        doctor_card.click_recording_button()

    with allure.step("Проверка заголовка формы"):
        expect(form.title_form).to_have_text("Запись на прием")

    with allure.step("Ввести данные"):
        form.fill_input_name("Зу12Ru")
        form.fill_input_telephone("00110011001")
        form.fill_input_date("11111-11-11")
        form.fill_input_comment("You need записаться a rheumatologist %& прием 275760
dates.")

    with allure.step("Нажать на кнопку “Записаться”"):
        form.click_button_recording()

    with allure.step("Проверка уведомлений о некорректно введенных данных фамилии и
имени "):
        expect(form.get_name_error()).to_have_text("Введены некорректные данные")

    with allure.step("Проверка уведомлений о некорректно введенных данных телефонного
номера"):
        expect(form.get_telephone_error()).to_have_text("Введён некорректный
телефонный номер")

    with allure.step("Проверка уведомлений о некорректно введенных данных даты записи
на прием "):
        expect(form.get_date_error()).to_have_text("Введена некорректная дата")
```

test_16_recording_a_doctor_incomplete_data.py

```
import allure
from playwright.sync_api import Page, expect
from components.cart_profile_doctor import CardProfileDoctor
from components.form_recording_a_doctor import FormRecording
from pages.page_nevrolog import PageNevrolog
```

```
@allure.title("Запись на приём на странице Неврология: отправка формы с неполными
данными — проверка валидации")
@allure.description("Форма записи не отправляется, если обязательные поля (имя,
телефон, дата) пусты")
def test_recording_a_doctor_incomplete_data(page: Page):
    nevrolog_page = PageNevrolog(page, "Врач-невролог клиники Alimma")
    doctor_name = "Тихонова Ольга Ивановна"
    doctor_card = CardProfileDoctor(page, doctor_name)
    form = FormRecording(page)
    nevrolog_page.open()

    with allure.step("Пролистать статью до секции- Врач-невролог клиники Alimma"):
        nevrolog_page.scroll_to_section()
        expect(nevrolog_page.section).to_have_text("Врач-невролог клиники Alimma")
        expect(doctor_card.wrapper).to_be_visible()
        page.wait_for_timeout(2000)

    with allure.step("Нажать кнопку “Записаться к врачу”, под анкетой врача"):
        doctor_card.click_recording_button()
        expect(form.title_form).to_have_text("Запись на прием")

    with allure.step("Заполнить форму не полными данными"):
        expect(form.title_form).to_be_visible()
        form.fill_input_name(",Mbuygt76")
        form.fill_input_telephone("")
        form.fill_input_date("")
        form.fill_input_comment("")

    with allure.step("Нажать кнопку “Записаться”, под анкетой врача"):
        form.click_button_recording()

    with allure.step("Проверка уведомлений о необходимости заполнить данные номер
тлефона"):
        expect(form.get_telephone_error()).to_have_text("Это поле является
обязательным.")

    with allure.step("Проверка уведомлений о необходимости заполнить данные даты
приема"):
        expect(form.get_date_error()).to_have_text("Это поле является обязательным.")
```

Components

base_component.py

```
from playwright.sync_api import Page, Locator
class BaseComponent:
    def __init__(self, page: Page, wrapper: Locator):
        self.page = page
        self.wrapper: Locator = wrapper

    def click(self):
        self.wrapper.click()
```

cart_doctor.py

```
from playwright.sync_api import Page
from components.base_component import BaseComponent
from controls.about_me import AboutMe
from pages.page_doctor_profile import DoctorProfile


class CartDoctor(BaseComponent):
    ''' Карточка доктора'''
    def __init__(self, page, doctor_name):
        self.doctor_name = doctor_name
        super().__init__(page, page.locator(f".team-box-layout2:has(.item-title a:has-
text('{doctor_name}'))"))
```

```
    def get_button_about_me(self):
        return AboutMe(self.page, self.wrapper)

    def click_button_about_me(self):
        self.get_button_about_me().wrapper.click()
```

cart_profile_doctor.py

```
from components.base_component import BaseComponent


class CardProfileDoctor(BaseComponent):
    ''' Карточка профиля доктора с данными'''
    def __init__(self, page, doctor_name):
        self.doctor_name = doctor_name
        super().__init__(page, page.locator(f".doctor-card:has(.doctor-
card__title:has-text('{doctor_name}'))"))

    def get_recording_button_by_name(self):
        return self.wrapper.locator("a:has-text('Записаться к врачу')")

    def click_recording_button(self):
        self.get_recording_button_by_name().click()
```

carusel_uslugi.py

```
from playwright.sync_api import Page
from components.base_component import BaseComponent
from controls.strelka_next_prew import SrtelkaNextPrew


class CaruselUslugi(BaseComponent):
    '''Карусель услуг на главной странице'''
    def __init__(self, page: Page):
        self.carusel = page.locator('#uslugi .rt-owl-carousel[data-carousel-options]')
        super().__init__(page, self.carusel)
        self.strelka = SrtelkaNextPrew(page, self.carusel)

    def get_title_uslugi(self):
        title = self.page.locator('.rt-el-title .rtin-title')
        return title.text_content().strip()

    def click__prev(self):
        self.strelka.click_prev()

    def click_next(self):
        self.strelka.click_next()

    def get_all_cart(self) -> list:
        return self.carusel.locator('.rtin-item').all()

    def click_cart_by_title(self, title: str):
        self.carusel.locator(f'.rtin-item:has(h3.item-title a:has-
text("{title}"))').first.click()

    def cart_visible_by_title(self, title: str):
        visible_cart = self.carusel.locator(f'.owl-item.active .rtin-item:has(h3.item-
title a:has-text("{title}"))')
        return visible_cart.count() > 0

    def scroll_to_item_by_title(self, title):
        for name in range(10):
            if self.cart_visible_by_title(title):
                return True
            self.click_next()
        return False

    def click_item_by_title(self, title: str):
        card = self.carusel.locator(f'.rtin-item:has(h3.item-title a:has-
text("{title}"))').first
        card.click()
```

form_recording_a_doctor.py

```
from playwright.sync_api import Page, Locator, expect
from components.base_component import BaseComponent
from controls.button_form import ButtonForm
from controls.input_form import InputForm


class FormRecording(BaseComponent):
    ''' Форма записи на прием'''

    def __init__(self, page: Page):

        wrapper = page.locator("#popmake-3802")
        super().__init__(page, wrapper)
        self.title_form = self.wrapper.locator(".pum-title")
        self.success_message = self.wrapper.locator(".form_success")
        self.name_input = InputField(self.wrapper.locator('input[name="your-name"]'))
        self.phone_input = InputField(self.wrapper.locator('input[name="tel-214"]'))
        self.date_input = InputField(self.wrapper.locator('input[name="date-464"]'))
        self.comment_input = InputField(self.wrapper.locator('textarea[name="textarea-
835"]'))

        self.submit_button = ButtonForm(page, self.wrapper)

    def fill_input_name(self, name: str):
        self.name_input.fill(name)

    def fill_input_telephone(self, phone: str):
        self.phone_input.fill(phone)

    def fill_input_date(self, date: str):
        self.date_input.fill(date)

    def fill_input_comment(self, text: str):
        self.comment_input.fill(text)

    def click_button_recording(self):
        self.submit_button.click_button_recording()

    def get_name_error(self) -> Locator:
        return self.name_input.get_error()

    def get_telephone_error(self) -> Locator:
        return self.phone_input.get_error()

    def get_date_error(self) -> Locator:
        return self.date_input.get_error()

    def check_title(self, title: str):
        expect(self.title_form).to_have_text(title)

    def get_success_message(self):
        expect(self.success_message.locator("p span")).to_have_text("Ваша заявка
успешно отправлена!")
        expect(self.success_message.locator("p").last).to_have_text("Наш менеджер
скоро с вами свяжется")

    def get_name_label(self) -> Locator:
        return self.wrapper.locator('label.form_name')

    def get_phone_label(self) -> Locator:
        return self.wrapper.locator('label.form_phone')

    def get_date_label(self) -> Locator:
        return self.wrapper.locator('label.form_date')
```

header.py

```
from playwright.sync_api import Page, Locator, expect
```

```
from components.base_component import BaseComponent
from components.logo_component import LogoHeader
from controls.link_menu import LinkMenu


class Header(BaseComponent):
    def __init__(self, page: Page):
        self.header = page.locator('#masthead')
        super().__init__(page, self.header)
        self.logo = LogoHeader(page)
        self.navigation = LinkMenu(page, self.header)

    def click_logo(self):
        self.logo.click()

    def get_menu_link(self, name: str, ) -> Locator:
        return self.navigation.get_menu_link(name)

    def hover_menu(self, name: str):
        self.navigation.hover_menu(name)

    def click_menu_link(self, name: str):
        self.navigation.click_menu_item(name)

    def click_submenu(self, parent_name: str, child_name: str):
        self.navigation.click_submenu(parent_name, child_name)
```

logo_component.py

```
from components.base_component import BaseComponent
from playwright.sync_api import Page


class LogoHeader(BaseComponent):
    '''Логотип в Header'''
    def __init__(self, page: Page):
        self.logo = page.locator('a.dark-logo')
        super().__init__(page, self.logo)

    def click_logo(self):
        self.logo.click()
```

window_messenger.py

```
from components.base_component import BaseComponent
from playwright.sync_api import Page, expect


class WindowMessager(BaseComponent):
    '''Окно, чат для связи'''
    def __init__(self, page: Page):
        self.window_chat = page.locator("#mvvwo_chat_box")
        super().__init__(page, self.window_chat)

    def get_title(self):
        return self.window_chat.locator("#mvvwo_chat_head")

    def get_status(self):
        return self.window_chat.locator(".mvvwo_online")

    def get_input(self):
        return self.window_chat.locator("#mvvwo_chatSend")

    def get_text_in_window(self):
        return self.window_chat.locator(".mvvwo_chat_body p")

    def get_button_close(self):
        return self.window_chat.locator("span.mvvwo_chat_close")

    def click_button_close(self):
        self.get_button_close().click()
```

```
    def get_send_button(self):
        return self.window_chat.locator("a#mvvwo_fab_send")

    def click_button_send(self):
        self.get_send_button().click()

    def clear_message(self, text: str):
        input_field = self.get_input()
        input_field.fill(text)
```

Controls

about_me.py

```
from playwright.sync_api import Page, Locator
from controls.base_control import BaseControl


class AboutMe(BaseControl):
    """Кнопка Обо мне"""
    def __init__(self, page: Page,
                 paren_wrapper: Locator,
                 self_wrapper_path='a.item-btn:has-text("Обо мне")'):
        super().__init__(page, paren_wrapper.locator(self_wrapper_path))
```

base_control.py

```
from playwright.sync_api import Page, Locator


class BaseControl:
    def __init__(self, page: Page, wrapper: Locator):
        self.page = page
        self.wrapper: Locator = wrapper

    def click(self):
        self.wrapper.click(force=True)
```

button_form.py

```
from playwright.sync_api import Page, Locator
from controls.base_control import BaseControl


class ButtonForm(BaseControl):
    ''' Кнопки "Записаться к врачу " и "Закрытие" в форме записи на прием'''
    def __init__(self, page: Page,
                 wrapper: Locator):
        super().__init__(page, wrapper)

    def get_button_recording(self):
        return self.wrapper.locator('button.g4_send_cf7_form')

    def click_button_recording(self):
        self.get_button_recording().click()

    def get_button_close(self):
        return self.wrapper.locator(".pum-close")

    def click_close(self):
        self.get_button_close().click()
```

input_form.py

```
from playwright.sync_api import Page, Locator
from controls.base_control import BaseControl


class InputForm(BaseControl):
    ''' Инпут в форме запись на прием '''
    def __init__(self, locator: Locator):
        super().__init__( locator.page, locator)
```

```
    def fill(self, value: str):
      self.wrapper.fill(value)

    def get_value(self):
        return self.wrapper.input_value()

    def get_error(self):
        return self.wrapper.locator('.wpcf7-not-valid-tip')
```

link_menu.py

```
from playwright.sync_api import Page, Locator, expect
from controls.base_control import BaseControl


class LinkMenu(BaseControl):
    '''Главное навигационное меню и подменю'''
    def __init__(self, page: Page, parent_locator: Locator):
        self.menu = parent_locator
        super().__init__(page, self.menu)

    def get_menu_link(self, name: str):
        return self.menu.get_by_role("link", name=name).first

    def get_menu_item(self, name: str):
        return self.menu.locator(f'li.menu-item:has(a:has-text("{name}"))').first

    def hover_menu(self, name: str):
        self.get_menu_link(name).hover()

    def click_menu_item(self, name: str):
        self.get_menu_link(name).click()

    def click_submenu(self, parent_name: str, child_name: str):
        parent_item = self.get_menu_item(parent_name)
        parent_item.hover()
        child_link = self.menu.get_by_role("link", name=child_name).first
        expect(child_link).to_be_visible(timeout=3000)
        child_link.click()
```

link_prodoctorov.py

```
from playwright.sync_api import Page, Locator
from controls.base_control import BaseControl


class LinkProdoctorov(BaseControl):
    '''Ссылка Prodoctorov'''

    def __init__(self, page: Page, parent_locator: Locator, name: str):
        self.name = name
        self.link = parent_locator.get_by_role("link", name="Prodoctorov")
        super().__init__(page, self.link)

    def click(self):
        self.link.click()
```

messenger_link.py

```
from playwright.sync_api import Page, Locator
from controls.base_control import BaseControl


class MessengerLink(BaseControl):
    ''' Ссылка-иконка для открытия чата'''

    def __init__(self, page: Page,
                 wrapper: Locator):
        super().__init__(page, wrapper)
```

```
    def get_link_messenger(self):
        return self.wrapper.locator("a.mvvwo_btn")

    def click_link_messenger(self):
        self.get_link_messenger().click()
```

pagination.py

```
from playwright.sync_api import Page, Locator
from controls.base_control import BaseControl


class Pagination(BaseControl):
    ''' Кнопки Пагинации '''

    def __init__(self, page: Page,
                 wrapper: Locator):
        super().__init__(page, wrapper)

    def get_button_prev(self):
        return self.wrapper.locator("li a:has(i.fa-angle-double-left)")

    def click_button_prev(self):
        self.get_button_prev().click()

    def get_button_next(self):
        return self.wrapper.locator("li a:has(i.fa-angle-double-right)")

    def click_button_next(self):
        self.get_button_next().click()

    def get_actual_number(self) -> int:
        active_li = self.wrapper.locator("li.active a")
        if active_li.count() > 0:
            return int(active_li.first.inner_text().strip())
        return 1

    def go_to_page(self, page_number: int):
        link = self.wrapper.locator(f"li a:has-text('{page_number}')")
        if link.count() > 0:
            link.first.click()
        else:
            raise Exception(f"Ссылка нас страницу {page_number} не найдена")

    def get_active_button(self):
        return self.wrapper.locator("li.active a")
```

strelka_next_prew.py

```
from playwright.sync_api import Page, Locator
from controls.base_control import BaseControl


class SrtelkaNextPrew(BaseControl):
    ''' Стрелки вперед, назад в карусели Услуги '''

    def __init__(self, page: Page,
                 wrapper: Locator):
        super().__init__(page, wrapper)
        self.button_prev = wrapper.locator('.owl-prev')
        self.button_next = wrapper.locator('.owl-next')

    def click_prev(self):
        self.button_prev.click()

    def click_next(self):
        self.button_next.click()
```

Pages

base_page.py

```
from playwright.sync_api import expect


class BasePage:
    def __init__(self, page, url):
        self.page = page
        self.url = url

    def open(self):
        return self.page.goto(self.url)

    def check_title_h1(self, title: str):
        expect(self.page.locator('h1.entry-title')).to_have_text(title)
```

main_page.py

```
from playwright.sync_api import expect
from components.carusel_uslugi import CaruselUslugi
from components.header import Header
from pages.base_page import BasePage
from controls.messenger_link import MessengerLink


class MainPage(BasePage):
    ''' Главная страница '''
    def __init__(self, page):
        self.carusel = CaruselUslugi
        self.url = "https://alimma-ufa.ru/"
        super().__init__(page, self.url)
        self.header = Header(page)
        self.messenger_link = MessengerLink(page, page.locator("body"))

    def open_main_page(self):
        self.page.goto(self.url)

    def get_header(self):
        return Header(self.page)

    def is_logo_visible(self) -> bool:
        header = self.get_header()
        return header.logo.wrapper.is_visible()

    def click_link_messenger(self):
        self.messenger_link.click_link_messenger()

    def check_title_h2(self, title: str):
        expect(self.page.get_by_role("heading", name=title,
level=2)).to_have_text(title)
```

page_bol_golenostopnom_sustave.py

```
from pages.base_page import BasePage


class RevmatologGolenostop(BasePage):
    ''' Страница Лечение боли в голеностопном сутставе в Уфе'''
    def __init__(self, page):
        url = "https://alimma-ufa.ru/uslugi/travmatologiya/bol-v-golenostopnom-
sustave/"
        super().__init__(page, url)
```

page_doctor_profile.py

```
from playwright.sync_api import Page
from pages.base_page import BasePage


class DoctorProfile(BasePage):
    def __init__(self, page: Page, url=None):
        super().__init__(page, url)
```

```
    def get_title_about_me(self):
        return self.page.locator("h3.section-title:has-text('Обо Мне:')")

    def get_text_about_me(self):
        return self.page.locator("div.single-item p[align='justify']")

    def get_title_diplom(self):
        return self.page.locator("h3.section-title:has-text('Дипломы И
Сертификаты:')")

    def diplom_table(self):
        return self.page.locator("table.table tbody tr")

    def diplom_table_year(self, index: int = 0):
        return self.page.locator(f"table.table tbody tr:nth-child({index + 1}) td:nth-
child(1)")

    def diplom_table_stepen(self, index: int = 0):
        return self.page.locator(f"table.table tbody tr:nth-child({index + 1}) td:nth-
child(2)")

    def diplom_table_university(self, index: int = 0):
        return self.page.locator(f"table.table tbody tr:nth-child({index + 1}) td:nth-
child(3)")

    def get_appointment_button(self):
        return self.page.locator("a.knopka:has-text('Записаться на прием')")
```

page_nevrolog.py

```
from playwright.sync_api import expect
from components.cart_profile_doctor import CardProfileDoctor
from pages.base_page import BasePage


class PageNevrolog(BasePage):
    ''' Страница Неврологии'''
    def __init__(self, page, section_text):
        url = "https://alimma-ufa.ru/uslugi/nevrologiya/"
        super().__init__(page, url)
        self.section = page.locator(f"h3.elementor-heading-title:has-
text('{section_text}')").first

    def get_section_title(self):
        return self.section.text_content()

    def scroll_to_section(self):
        self.section.scroll_into_view_if_needed()
        expect(self.section).to_be_visible()

    def get_doctor_card(self, doctor_name: str):
        return CardProfileDoctor(self.page, doctor_name)
```

page_price_uslug.py

```
from pages.base_page import BasePage


class PriceUslug(BasePage):
    ''' Страница Цен клиники Alimma '''

    def __init__(self, page):
        url = "https://alimma-ufa.ru/price/"
        super().__init__(page, url)

    def click_section(self, section_name: str):
        section = self.page.locator(f'div.elementor-tab-title:has(a.elementor-toggle-
title:has-text("{section_name}"))')
        section.scroll_into_view_if_needed()
        section.wait_for(state='visible', timeout=5000)
        section.click()
```

```
    def get_table(self):
        table = self.page.locator('div.elementor-tab-content.elementor-active
table.tbl')
        return table.is_visible()

    def get_table_code(self, code: str):
        return self.page.locator(f'table.tbl tr:has(td.tbl-kod:has-text("{code}"))')

    def get_table_name_uslugi(self, name: str):
        return self.page.locator(f'table.tbl tr:has(td.tbl-naz:has-text("{name}"))')

    def get_table_price(self, price: str):
        return self.page.locator(f'table.tbl tr:has(td.tbl-cena:has-text("{price}"))')

    def get_price_by_code(self, code: str):
        row = self.get_table_code(code)
        return row.locator("td.tbl-cena")

    def get_price_by_name(self, name: str):
        row = self.get_table_name_uslugi(name)
        return row.locator("td.tbl-cena")
```

page_prodoctorov.py

```
import re
from playwright.sync_api import expect
from pages.base_page import BasePage


class PageProdoctorov(BasePage):
    ''' Страница с отзывами на Prodoctorov '''
    def __init__(self, page):
        super().__init__(page, url='')

    def check_is_prodoctorov(self):

expect(self.page).to_have_url(re.compile(r"https?://(www\.)?prodoctorov\.ru/.*"))
```

page_revmatolog.py

```
from pages.page_doctor_profile import DoctorProfile
from pages.base_page import BasePage
from components.cart_doctor import CartDoctor


class PageRevmatolog(BasePage):
    ''' Страница категории - Ревматолог'''
    def __init__(self, page):
        url = "https://alimma-ufa.ru/medilink_doctor_category/revmatolog/"
        super().__init__(page, url)

    def get_name_doctor(self, name_doctor: str):
        return CartDoctor(self.page, name_doctor)

    def click_about_me(self, name_doctor: str):
        doctor = self.get_name_doctor(name_doctor)
        doctor.click_button_about_me()
        return DoctorProfile(self.page)

    def click_about_me_doctor(self, doctor_name: str):
        return self.click_about_me(doctor_name)

    def get_doctor_cards_count(self) -> int:
        cards = self.page.locator(".team-box-layout2")
        return cards.count()

    def get_all_doctor_cards(self):
        return self.page.locator(".team-box-layout2")

    def get_doctor_cards(self):
        return self.page.locator(".team-box-layout2")
```

page_speczialisty.py

```
from pages.base_page import BasePage


class PageSpeczialisty(BasePage):
    ''' Страница Специалисы клиники Alimma '''
    def __init__(self, page):
        url = "https://alimma-ufa.ru/speczialisty-kliniki/"
        super().__init__(page, url)

    def get_doctors_list(self):
        return self.page.locator('.doctor-item').all()

    def get_doctor_count(self):
        return self.page.locator('.doctor-item').count()
```

page_uslugi.py

```
from components.header import Header
from controls.pagination import Pagination
from pages.base_page import BasePage


class PageUslugi(BasePage):
    ''' Страница Услуги клиники Alimma '''

    def __init__(self, page):
        url = "https://alimma-ufa.ru/uslugi/"
        super().__init__(page, url)
        self.header = Header(page)
        pagination_wrapper = self.page.locator("div.pagination-area").first
        self.pagination = Pagination(page, pagination_wrapper)

    def get_uslugi(self):
        return self.page.locator(".rtin-item")

    def get_uslugi_all(self) -> list:
        return self.get_uslugi().all()

    def get_name_uslugi(self) -> list:
        items = self.get_uslugi_all()
        names = []
        for item in items:
            title_elem = item.locator("h3.item-title a")
            if title_elem.count() > 0:
                names.append(title_elem.first.inner_text().strip())
        return names

    def count_uslugi(self) -> int:
        return len(self.get_name_uslugi())

    def click_number(self, number: int):
        self.pagination.go_to_page(number)
```